



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, ethereal sea of clouds. The person is looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and yellow. A bright, glowing light source, possibly a sun or moon, is visible on the horizon, casting a warm glow. The sky is also filled with stars, a comet streaking across the upper left, and a large, colorful nebula or galaxy in the upper right. The overall atmosphere is one of wonder and exploration.

Marco Servetto
CO258

www.youtube.com/MarcoServetto

Generics

Truck<>





Generics



What are generics?

- Introduced in Java 1.5
- Before Java generics:
 - **Can only say things like: 'cs' is a List of Objects**
 - Then, can put any Object into 'cs' without restriction
 - Get Objects out of cs – unsafe casts needed recover their cat-ness.
- With Java Generics:
 - **Can say things like: 'cs' is a List of Cats**
 - Then, can only put Cats into 'cs'
 - Directly get Cats out of 'cs' – no casting required!

Why Generics?

```
class List {  
    private Object[] elems= new Object[16];  
    private int end= 0;  
    public void add(Object e) {  
        if (end == elems.length) { /*allocate more space*/ }  
        elems[end] = e;  
        end += 1;  
    }  
    public Object get(int index) {  
        if (index >= end) { throw ... }  
        else { return elems[index]; }  
    }  
}
```

```
List cs= new List();  
cs.add(new Cat());  
Cat c= (Cat) cs.get(0); // have to cast :-(  

```

This says cs is a List
of Objects

We know this returns
a Cat, but we still have
to cast

How can we say cs is a
List of Cats?

The Generic version

```
class List<T> {  
    private T[] elems= (T[]) new Object[16];  
    private int end= 0;  
    public void add( T e ) {  
        if (end == elems.length) { ... }  
        elems[end] = e;  
        end += 1;  
    }  
    public T get( int index ) {  
        if( index >= end ) { throw ... }  
        else { return elems[index]; }  
    }  
}
```

"T" is a generic
type parameter

"T" represents the
type of object held
in List

This says v is a Vec
of Cats

Can only put Cats
into v

```
List<Cat> cs= new List<Cat>();  
cs.add(new Cat());  
Cat c= cs.get(0); // don't have to cast :-)
```

Can only get Cats out
of v

Shape Example

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

```
class ShapeGroup implements Shape {  
    private List shapes= new ArrayList();  
    ...
```

```
    public void draw(Graphics2D g) {  
        for(Shape s : shapes) {  
            s.draw(g);  
        }  
    }  
}
```

- Q) Why doesn't this compile ?

Shape Example

A

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

B

```
class ShapeGroup implements Shape {
```

C

```
    private List shapes= new ArrayList();
```

```
    ...
```

D

```
    public void draw(Graphics2D g) {
```

E

```
        for(Shape s : shapes) {
```

```
            s.draw(g);
```

```
        }
```

```
    }
```

```
}
```

- Q) Why doesn't this compile ?

Shape Example

A

```
interface Shape { void draw(Graphics2D g); }
```

B

```
class Square implements Shape { ... }
```

C

```
class ShapeGroup implements Shape {  
    private List shapes= new ArrayList();  
    ...
```

D

```
    public void draw(Graphics2D g) {  
        for(Shape s : shapes) {  
            s.draw(g);  
        }
```

E

```
    }  
}
```

```
ShapeGroup.java:7: incompatible types  
found   : java.lang.Object  
required: Shape  
        for(Shape s : shapes) {  
                           ^  
1 error
```

- Q) Why doesn't this compile ?

Using Generics in Shape

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

```
class ShapeGroup implements Shape {  
    private List<Shape> shapes= new ArrayList<Shape>();  
    ...  
  
    public void draw(Graphics2D g) {  
        for(Shape s : shapes) {  
            s.draw(g);  
        }  
    }  
}
```

Generic ShapeGroup ?

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

```
class ShapeGroup<T> implements Shape {  
    private List<T> shapes= new ArrayList<T>();  
    ...  
  
    public void draw(Graphics2D g) {  
        for(T s : shapes) {  
            s.draw(g);  
        }  
    }  
}
```

Generic ShapeGroup ?

A

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

B

```
class ShapeGroup<T> implements Shape {
```

C

```
    private List<T> shapes= new ArrayList<T>();
```

```
    ...
```

D

```
    public void draw(Graphics2D g) {
```

E

```
        for(T s : shapes) {
```

```
            s.draw(g);
```

```
        }
```

```
    }
```

```
}
```

- Q) Now what's wrong?

Generic ShapeGroup ?

A

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

B

```
class ShapeGroup<T> implements Shape {
```

C

```
    private List<T> shapes= new ArrayList<T>();
```

```
    ...
```

D

```
    public void draw(Graphics2D g) {
```

E

```
        for(T s : shapes) {
```

```
            s.draw(g);
```

```
        }
```

```
    }
```

```
}
```

are we sure T
has a draw()
method ???



- Q) Now what's wrong?

Type Bounds

- Upper Bound on Generic Type:

```
<T extends Shape>
```

- “T is a generic parameter which must extend Shape”

Generic ShapeGroup

A

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

B

```
class ShapeGroup<T extends Shape> implements Shape {
```

C

```
    private List<T> shapes= new ArrayList<T>();
```

```
    ...
```

D

```
    public void draw(Graphics2D g) {
```

E

```
        for(T s : shapes) {
```

```
            s.draw(g);
```

```
        }
```

```
    }
```

```
}
```

F: it is fine

Using Generic ShapeGroup

A:ok
B:error

```
public static void main(String[] args) {  
    ShapeGroup<Square> sg1= new ShapeGroup<Square>();  
    sg1.add(new Square(...));  
  
    ShapeGroup<String> sg2= new ShapeGroup<String>();  
    sg2.add("Hello World");  
}  
...  
class Foo<T> {  
    private ShapeGroup<T> group;  
    ...  
}
```

- Spot the errors!!

Using Generic ShapeGroup

```
public static void main(String[] args) {  
    ShapeGroup<Square> sg1= new ShapeGroup<Square>();  
    sg1.add(new Square(...));
```

```
    ShapeGroup<String> sg2= new ShapeGroup<String>();  
    sg2.add("Hello World");
```

```
}
```

```
...
```

```
class Foo<T> {  
    private ShapeGroup<T> group;
```

```
    ...
```

```
}
```

A:ok
B:error

- Spot the errors!!

Using Generic ShapeGroup

```
public static void main(String[] args) {  
    ShapeGroup<Square> sg1= new ShapeGroup<Square>();  
    sg1.add(new Square(...));  
  
    ShapeGroup<String> sg2= new ShapeGroup<String>();  
    sg2.add("Hello World");  
}
```

...

```
class Foo<T> {  
    private ShapeGroup<T> group;  
    ...  
}
```

A:ok
B:error

- Spot the errors!!

Exact use:

```
<T extends Type>
```

- *Type* have to be the name of class or interface

Exact use:

`<T extends T1 & T2 ...>`

- You can provide more than one!

Exact use:

```
<T1, T2 extends List<T1>>
```

- You can express non trivial ones!

Generics: intuition

```
class A<T>{...} == class A<T extends Object>{...}
```

```
class A<B extends C>{...}
```

‘B’ is a ‘type variable’ of kind ‘C’

This is similar to how method parameters works!

When we write

A<Foo> it is like calling a method and passing ‘Foo’ as the value for the ‘local variable’ B.

As for method calls, this works only if Foo is a valid example of a ‘C’.

Generic ShapeGroup, alternative

```
interface Shape { void draw(Graphics2D g); }
```

```
class Square implements Shape { ... }
```

```
abstract class ShapeGroup<T> implements Shape {  
    abstract void draw(T t, Graphics2D g);  
    private List<T> shapes= new ArrayList<T>();  
    ...  
    public void draw(Graphics2D g) {  
        for(T s : shapes) {  
            draw(s, g);  
        }  
    }  
}
```

```
//Here the user must extend ShapeGroup  
//and implement draw. Verbose, but flexible.
```



Using the alternative Generic ShapeGroup

//To use it with Square, it is a little more verbose.

//Use anonymous inner class to satisfy draw with t.draw

```
ShapeGroup<Square> sg1= new ShapeGroup<Square>(){  
    void draw(Square t, Graphics2D g){ return t.draw(g); }  
};  
sg1.add(new Square(...));
```

//But now, it can be used with String too, or anything else really.

//Use anonymous inner class to chose how to draw stuff

```
ShapeGroup<String> sg2= new ShapeGroup<String>(){  
    void draw(String t, Graphics2D g){ return g.drawString(t,10,10); }  
};  
sg2.add("Hello World");
```



Generic Classes
v.s.
Generic Methods

Generic Methods

- How to write `min()` method for subclasses of `Point`?
- How to write an auxiliary class with a method that, taking two `Point` instances, returns the one nearest the origin (0,0)

Generic Methods

- How to write min() method for subclasses of Point?
 - Should be possible since subclasses all have x and y fields

```
class Point{ int x; int y; ...}
class ColPoint extends Point{ int colour; ...}

class Geometry{
    Point min(Point a, Point b) {
        var aMin = a.x < b.x || (a.x == b.x && a.y < b.y);
        if (aMin){ return a; }
        return b;
    }
    void user(){
        ColPoint c1= new ColPoint(...);
        ColPoint c2= new ColPoint(...);

        c1 = min(c1,c2);
    }
}
```

Error or correct?

Generic Methods

- How to write min() method for subclasses of Point?
 - Should be possible since subclasses all have x and y fields

```
class Point{ int x; int y; ...}
class ColPoint extends Point{ int colour; ...}

class Geometry{
    Point min(Point a, Point b) {
        var aMin = a.x < b.x || (a.x == b.x && a.y < b.y);
        if (aMin){ return a; }
        return b;
    }
    void user(){
        ColPoint c1= new ColPoint(...);
        ColPoint c2= new ColPoint(...);

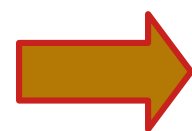
        c1 = (ColPoint) min(c1,c2);
    }
}
```

Needs cast on the return value!

Generic Methods

- How to write min() method for subclasses of Point?
 - Should be possible since subclasses all have x and y fields

```
class Point{ int x; int y; ...}  
class ColPoint extends Point{ int colour; ...}
```



```
class Geometry{  
    <T extends Point> T min(T a, T b) {  
        var aMin = a.x < b.x || (a.x == b.x && a.y < b.y);  
        if (aMin){ return a; }  
        return b;  
    }  
    void user(){  
        ColPoint c1= new ColPoint(...);  
        ColPoint c2= new ColPoint(...);  
  
        c1 = min(c1,c2);  
    }  
}
```

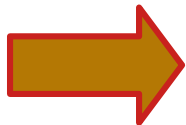
Generic Methods

- How to write min() method for subclasses of Point?
 - Should be possible since subclasses all have x and y fields

```
class Point{ int x; int y; ...}
class ColPoint extends Point{ int colour; ...}

class Geometry{
    <T extends Point> T min(T a, T b) {
        var aMin = a.x < b.x || (a.x == b.x && a.y < b.y);
        if (aMin){ return a; }
        return b;
    }
    void user(){
        ColPoint c1= new ColPoint(...);
        ColPoint c2= new ColPoint(...);

        c1 = min(c1,c2);
    }
}
```



Call site method generic parameters are inferred

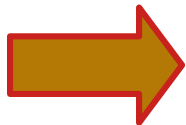
Generic Methods

- How to write min() method for subclasses of Point?
 - Should be possible since subclasses all have x and y fields

```
class Point{ int x; int y; ...}
class ColPoint extends Point{ int colour; ...}

class Geometry{
    <T extends Point> T min(T a, T b) {
        var aMin = a.x < b.x || (a.x == b.x && a.y < b.y);
        if (aMin){ return a; }
        return b;
    }
    void user(){
        ColPoint c1= new ColPoint(...);
        ColPoint c2= new ColPoint(...);

        c1 = this.<ColPoint>min(c1,c2);
    }
}
```



Call site method generic parameters are inferred

Type Erasure!

- Java Virtual Machine doesn't know about Generics
- They are enforced only at compile time
- Can't do **instanceof** on a generic, or **new instance**
- Can break the rules with casting ...

```
int sum(ArrayList<Integer> is) {  
    int r= 0;  
    for(Integer i : is) { r = r + i; }  
    return r;  
}
```

```
void foo(){  
    ArrayList list= new ArrayList();    //RAW TYPE warning, error in the WAT  
    list.add("Crash Me!");  
    int r = sum( (ArrayList<Integer>)list );  
}
```

A Var type

- In modern Java there are a few points where we need something to be (effectively-)final.
- We can circumvent those limitations by defining our own 'Var' type

```
final class Var<T>{  
    private T f;  
    public Var(T f){ this.f= f; }  
    T get(){ return f; }  
    void set(T f){ this.f = f; }  
    @Override public String toString(){ return f.toString(); }  
    @Override public int hashCode(){ return f.hashCode(); }  
    @Override public boolean equals(Object o){  
        return o instanceof Var<?> v && this.f.equals(v.f);  
    }  
}
```

- Now we can define records with 'updatable' fields
`record Person(Var<String> name, Var<Integer> age){}`

A Var type

```
record Person(Var<String> name, Var<Integer> age){}
```

- Is this good? Is this bad?
- We do not know.
- Some Java gurus think this is a terrible and misleading idea.
- Some Java gurus are greatly in favor of this.
 - If we embrace this fully, we may have a programming style where there is a single non final field in the whole program (the one inside Var) and every source of mutation has to pass through this field.
- To simplify learning, If I had to teach Java first year, I may teach Var and records and not mention the keyword 'class' until second year.

Next time?

